

構造体・クラス

構造体・クラスの前に、、、

C++, C#にはオブジェクト指向という考え方があります。

今の皆さんのパイザのスク립トは全部main関数の中に書き、順番にひたすら処理をするような書きかたしてます。

構造体・クラスの前に、、、

でも、よく考えてみてください。

Unityでコードを書く時に全部 1 個で書きますか？

PlayerならPlayer,EnemyならEnemyとイメージしやすい範囲で、プログラムファイルを区切って作成しますよね。

構造体・クラスの前に、、、

それがオブジェクト指向です。

人間が認識しやすいオブジェクト・物体を区切りにして、
データと処理をまとめる事でプログラムを書きやすくします。

パイザでもそうしましょう。

構造体とは

構造体とはデータの塊の事です。
変数をたくさんまとめて一つのものとして扱うことが出来ます。

2 個の参照

```
public struct Status
{
    public string name;
    public int hp;
    public int atk;
    public int def;
}
```

0 個の参照

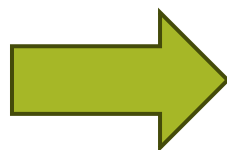
```
static void Main(string[] args)
{
    Status playerStatus = new Status();
    playerStatus.hp = 10;
}
```

構造体のメリット

たくさんの変数名を置く必要がなくなります。

```
static void Main(string[] args)
{
    string playerName = "";
    int playerHp = 0;
    int playerAtk = 0;
    int playerDef = 0;

    string enemyName = "";
    int enemyHp = 0;
    int enemyAtk = 0;
    int enemyDef = 0;
}
```



0 個の参照

```
static void Main(string[] args)
{
    Status playerStatus = new Status();
    Status enemyStatus = new Status();
}
```

構造体のメリット

```
static void Main(string[] args)
{
    string playerName = "";
    int playerHp = 0;
    int playerAtk = 0;
    int playerDef = 0;

    string enemyName = "";
    int enemyHp = 0;
    int enemyAtk = 0;
    int enemyDef = 0;
}
```

このような書き方をしていると、playerHpを変更しようとしてenemyHpを操作しちゃう。なんて事が多々あります。

前にも話しましたが、勘違いしにくく、ミスしにくい書き方は良い書き方です。

クラスとは

構造体とはデータの塊と処理がセットになったものの事です。

4 個の参照

```
public class Unit
{
    private string name = "";
    private int hp;
    private int atk;
    private int def;

    0 個の参照
    public void Attack() { }
    0 個の参照
    public void Hit() { }
}
```

0 個の参照

```
static void Main(string[] args)
{
    Unit player = new Unit();
    Unit enemy = new Unit();
}
```


クラスとは

構造体とはデータの塊と処理がセットになったものの事です。

4 個の参照

```
public class Unit
{
    private string name = "";
    private int hp;
    private int atk;
    private int def;

    0 個の参照
    public void Attack() { }
    0 個の参照
    public void Hit() { }
}
```

0 個の参照

```
static void Main(string[] args)
{
    Unit player = new Unit();
    Unit enemy = new Unit();
}
```

クラスとは

構造体と違って変数をできる限りPrivateで作成し、
基本的には呼び出された関数に沿ってのみ動作をさせます。

```
static void Main(string[] args)
{
    Unit player = new Unit();
    Status enemy = new Status();

    player.Hit();
    enemy.hp -= 10;
}
```



クラスに対しては関数の呼び出し
構造体に対しては直接変数を操作する

クラスと構造体の違い

関数の引数にされて、その関数の中で値を変えた際に変更が反映されるかされないかがクラスと構造体で違います。

0 個の参照

```
public void UnitDamage(Unit unit)
{
    // 呼び出し元のunitのデータにも反映される
    unit.Hit();
}
```

0 個の参照

```
public void StatusDamage(Status status)
{
    // 呼び出し元のunitのデータには反映されない
    status.hp--;

    //transform.position.x--;
    //が反映されないのも実はこの仕様の影響
}
```

問題

文字と整数の組のソート C#編

文字と整数の組のソート 2 C#編

静的メンバ C#編